



UNIVERSIDAD TECNOLÓGICA NACIONAL

FACULTAD REGIONAL TUCUMÁN

Trabajo Final Integrador

21/06/2026

Materia: Virtualización: Consolidación de Servidores

Comisión: 5K3

Profesores:

- Ing. Carriles, Luis Maria

Autor

- Hamada, Santino - 56070

Índice

1. Introducción	2
2. Inventario de Infraestructura (Proxmox VE)	2
3. Configuración y Optimización de la Base de Datos (LXC 144)	2
3.1. Optimización de Memoria (postgresql.conf)	2
3.2. Políticas de Seguridad de Red (pg_hba.conf)	3
4. Estrategia de Despliegue del Frontend (LXC 143)	3
4.1. Migración de Gestor de Paquetes (npm)	3
4.2. Compilación y Empaquetado Standalone	3
4.3. Justificación del Destino y Análisis de Descompresión	4
4.4. Secuencia de Despliegue en el Servidor (Consola LXC 143)	4
5. Conclusión	5

1. Introducción

El presente informe detalla la arquitectura, configuración e implementación de una aplicación web modular (Blog Personal) desplegada sobre la plataforma de virtualización de la nube académica NAP. El principal desafío del proyecto radica en operar bajo restricciones estrictas de hardware, asignando un límite máximo de **128 MB de memoria RAM** por nodo de servicio. Para mitigar esta limitación, se optó por una arquitectura desacoplada utilizando contenedores Linux (*LXC*) independientes sobre una distribución base eficiente.

2. Inventario de Infraestructura (Proxmox VE)

La topología de red local dentro del entorno virtualizado se compone de dos contenedores LXC aprovisionados con la plantilla oficial `debian-12-standard`, seleccionada por su bajo consumo de recursos en reposo (15-20 MB de RAM nativa).

Cuadro 1: Inventario de Contenedores LXC

LXC ID	Servicio	Rol	Límite RAM	Dirección IP
143	Next.js Standalone	Frontend / API	128 MB	172.16.90.143
144	PostgreSQL 15	Base de Datos	128 MB	172.16.90.144

3. Configuración y Optimización de la Base de Datos (LXC 144)

Debido al límite estricto de memoria, el motor PostgreSQL debió ser parametrizado exhaustivamente para evitar el colapso del servicio provocado por el *Out Of Memory (OOM) Killer* del kernel Linux.

3.1. Optimización de Memoria (postgresql.conf)

Se modificaron las variables críticas de asignación de memoria dentro de `/etc/postgresql/15/main` para garantizar que el búfer compartido y los procesos de trabajo operen holgadamente dentro de los 128 MB disponibles:

```
1 # Archivo: postgresql.conf
2 listen_addresses = '*'          # Habilita la escucha en interfaces
   de red virtualizadas
3 shared_buffers = 10MB          # Reducido dr sticamente para
   evitar sobrepasar la RAM del LXC
4 work_mem = 1MB                 # Memoria interna para operaciones
   de ordenamiento b sico
5 maintenance_work_mem = 8MB     # Limite estricto para tareas de
   mantenimiento (vacuum, etc.)
```

Listing 1: Parámetros de memoria optimizados

3.2. Políticas de Seguridad de Red (pg_hba.conf)

Para blindar el acceso a los datos y cumplir con las directivas de seguridad requeridas, se modificó el archivo de autenticación `pg_hba.conf`. Se implementó de manera estricta una máscara de red /32, permitiendo de forma exclusiva la conexión entrante originada desde la IP específica del contenedor del Frontend (LXC 143):

```
1 # Archivo: pg_hba.conf
2 # TYPE DATABASE USER ADDRESS
3 # METHOD
4 host blogdb postgres 172.16.90.143/32
5 scram-sha-256
```

Listing 2: Restricción estricta de origen

Justificación técnica: El uso de la máscara /32 en lugar del segmento genérico /24 restringe el acceso al socket TCP de la base de datos únicamente al nodo legítimo del frontend, bloqueando intentos de conexión cruzada desde otros contenedores vecinos del laboratorio.

4. Estrategia de Despliegue del Frontend (LXC 143)

4.1. Migración de Gestor de Paquetes (npm)

Durante las fases de pruebas iniciales se detectaron fallos críticos de tipo `MODULE_NOT_FOUND` provocados por la estructura de enlaces simbólicos (*symlinks*) nativa de `pnpm` al empaquetarse desde entornos Windows hacia Linux. Para solucionar este conflicto de manera definitiva, se realizó una migración táctica hacia `npm`, garantizando la compilación física y real de todos los módulos binarios dentro de la estructura de producción.

4.2. Compilación y Empaquetado Standalone

Para minimizar la huella de memoria en el contenedor de Next.js, se activó la opción `output: 'standalone'` en el archivo `next.config.js`. Esto genera un servidor Node.js minimalista que no requiere dependencias de desarrollo. El proceso de empaquetado y transferencia desde la máquina local hacia el contenedor LXC aislado de internet se estructuró mediante comandos estándar de compresión:

```
1 # Inyección de archivos estáticos requeridos por el servidor
2 standalone
3 xcopy /E /I /Y public .next\standalone\public
4 xcopy /E /I /Y .next\static .next\standalone\.next\static
5
6 # Compresión del bloque ejecutable libre de symlinks rotos
7 cd .next\standalone
8 tar -czf ../../blog_npm.tar.gz .
9 cd ../../
10
11 # Transferencia directa por canal seguro SCP al contenedor destino
12 de la NAP
```

```
11 scp blog_npm.tar.gz root@172.16.90.143:/tmp/
```

Listing 3: Comandos de preparación y empaquetado en entorno local

4.3. Justificación del Destino y Análisis de Descompresión

El archivo comprimido se transfiere directamente al directorio `/tmp/` del contenedor LXC 143. Al tratarse de una partición volátil del sistema, se mitigan posibles conflictos de permisos SSH y se evita la saturación de almacenamiento residual en el directorio raíz.

Para la extracción de la aplicación se ejecuta el comando `tar -xzf`, cuyas banderas operativas se justifican técnicamente a continuación:

- **-x (Extract):** Instruye al binario a ejecutar la función de desempaquetado de estructuras sobre el flujo de entrada.
- **-z (gzip):** Indica que el archivo posee una compresión basada en el algoritmo *gzip*, forzando la descompresión algorítmica previa a la lectura de datos.
- **-f (File):** Especifica la ruta física del archivo que actuará como origen del stream de datos (`/tmp/blog_npm.tar.gz`).

4.4. Secuencia de Despliegue en el Servidor (Consola LXC 143)

Inmediatamente después de que la transferencia por red finaliza con éxito, se ejecuta la siguiente secuencia de comandos en la terminal del contenedor web para aislar la instalación y configurar el entorno de producción:

```
1 # 1. Limpieza preventiva y creaci n del directorio de producci n
2 rm -rf /opt/blog
3 mkdir -p /opt/blog
4 cd /opt/blog
5
6 # 2. Descompresi n del flujo en la ruta actual
7 tar -xzf /tmp/blog_npm.tar.gz
8
9 # 3. Optimizaci n del almacenamiento borrando el archivo temporal
10 rm /tmp/blog_npm.tar.gz
11
12 # 4. Creaci n y persistencia de las variables de entorno de red
13 nano .env
```

Listing 4: Secuencia operativa de descompresión y despliegue local

Dentro del archivo `.env` se configuran las credenciales seguras y el direccionamiento TCP apuntando estrictamente al nodo de datos:

```
1 DB_HOST=172.16.90.144
2 DB_PORT=5432
3 DB_NAME=blogdb
4 DB_USER=postgres
5 DB_PASSWORD=45275660
6 PORT=3000
```

Finalmente, el servidor se inicializa en un entorno aislado de dependencias globales mediante el motor de ejecución nativo:

```
1 NODE_ENV=production node --env-file=.env server.js
```

5. Conclusión

La implementación exitosa de esta arquitectura demuestra que es viable desplegar aplicaciones modernas basadas en marcos de trabajo robustos como Next.js junto a sistemas de gestión de bases de datos relacionales robustos como PostgreSQL bajo escenarios de recursos extremadamente limitados (128 MB RAM). El correcto aplanamiento de módulos mediante **npm**, la extracción selectiva del modo *standalone* y la parametrización fina del búfer de base de datos constituyen la solución de ingeniería óptima para entornos académicos aislados.